

Introduction to Artificial Intelligence

Lecture 8: Language models I

September 29, 2025



Starting an AI project

How do you use machine learning to build an AI project?

- Workflow of AI projects: what is the workflow of an AI project?
- Selecting AI projects: How do you select an AI project? (either by yourself or as part of a team)
- Organizing data and team for the projects



Workflow of a machine learning project

- Example: speech recognition



Amazon alexa



Google home



Hey Siri

iPhone siri

Key steps of a machine learning project (speech recognition)

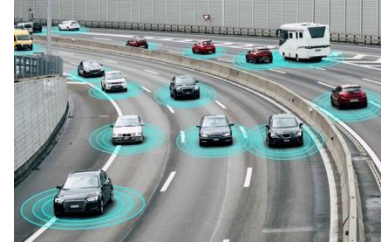
1. Collect data: get a people to say alexa, hello, etc (audio clips)
2. Train model: use a machine learning algorithm to learn input -> output mappings (audio #1 / #2 -> “alexa,” “hello”)
 - Iterate many times until good enough
3. Deploy model: put the AI software into a speaker and ship it into a group of users
 - You start to get new data back after shipping the product
 - Maintain / update model: Trained on English speakers and ship to Chinese-English speakers (working not as well -> update model with new data)



Workflow of a machine learning project

Self-driving car

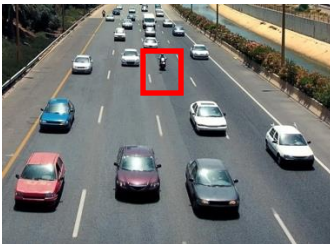
1. Collect data: image $\rightarrow$ position of other cars



2. Training model: Iterate many times until good enough

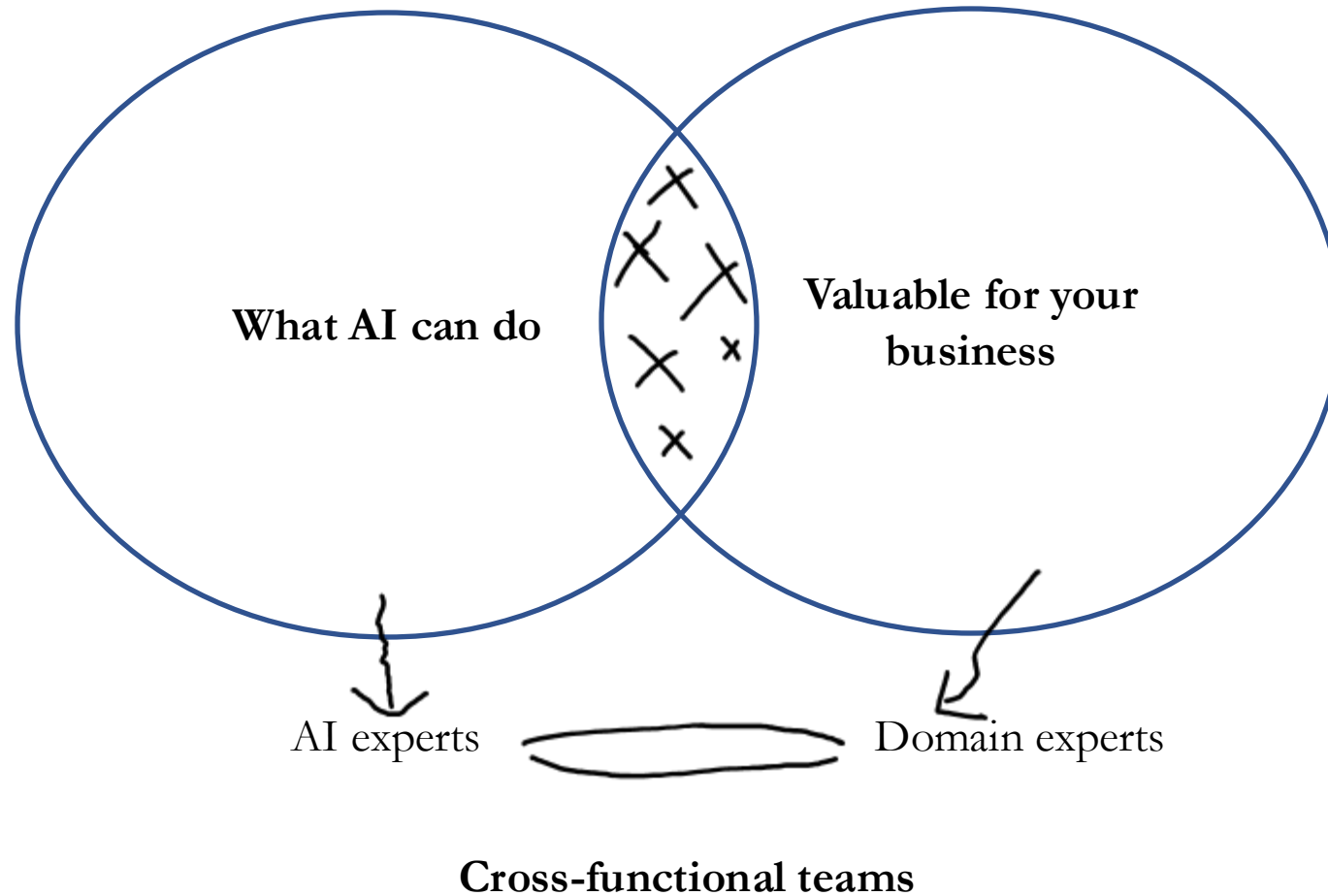


3. Deploy model: Get data back, maintain / update model (safety is #1)



How to choose an AI project

- AI knowledge and domain knowledge



Brainstorming frameworks

- Think about automating tasks rather than automating jobs
 - E.g., customer service call routing
 - Radiologists: reading xray scans, providing customized treatment for patients, mentoring junior radiologists, consulting for other doctors
- What are the main drivers of business value? A common question from a business executive point of view
- What are the main pain points in your business? Another valuable question to ask (some can be solved with AI, some cannot)

How AI is transforming healthcare: <https://www.oracle.com/health/ai-healthcare/>.

Issues: doctor trust, privacy rules, data quality, and data silos



You can make progress even without big data

- Having more data almost never hurts
 - Simple distinction between “big” vs. “small” data: can’t be analyzed on a single machine
 - Advances in technology means even “small” data is getting bigger: 32GB of RAM on laptops
- Data makes some businesses (like web search) defensible
 - Web search is a long-tail business: there are a lot of very rare queries, knowing these long-tail cases helps build a much better business
- A small dataset can go a long way:



Recap: Illustrating the weight decay algorithm

- Linear model: $Y = \beta_0 + X_1\beta_1 + X_2\beta_2 + \cdots + X_p\beta_p + \varepsilon$
- Number of parameters $p > n$ (number of observations) --- a very common training scenario in deep learning
- How can we regularize the parameter vector β to avoid overfitting?



Recap: Example

- Predict Boston house prices: Suppose we only have one observation ($n = 1$)

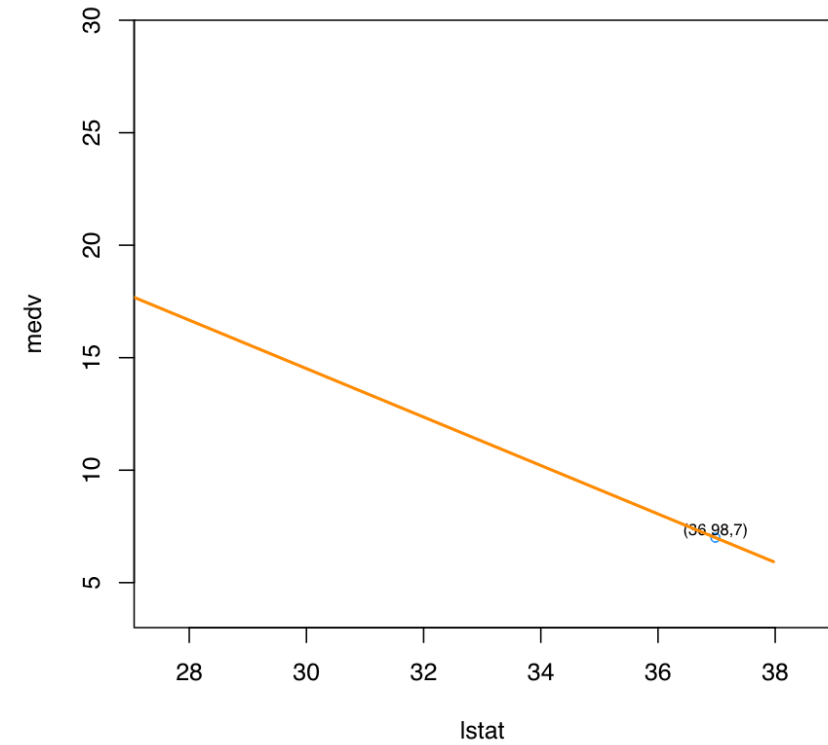
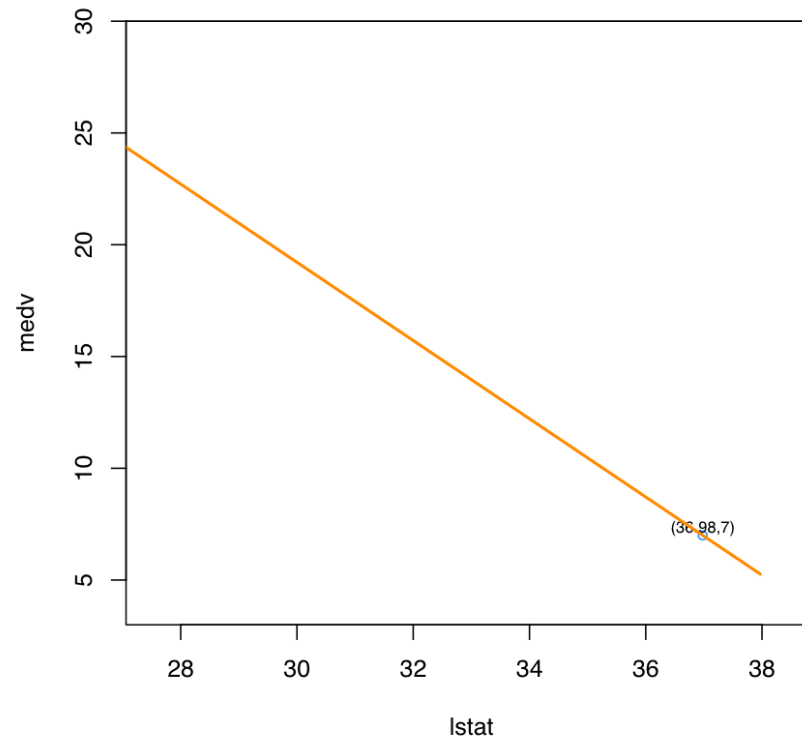
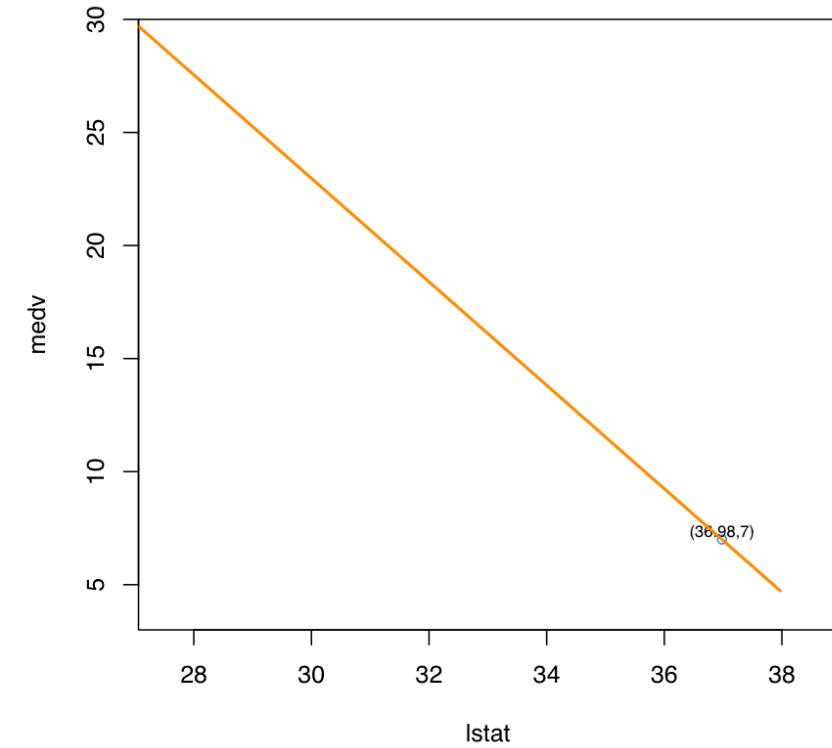
crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	lstat	medv
45.7461	0	18.1	0	0.693	4.519	100	1.6582	24	666	20.2	36.98	7

- Suppose we want to estimate the coefficients in simple linear regression:

$$medv = \beta_0 + lstat \cdot \beta_1 + \varepsilon$$



Recap: Choosing β_0 and β_1



All of these are valid solutions!



Recap: If we have one more observation

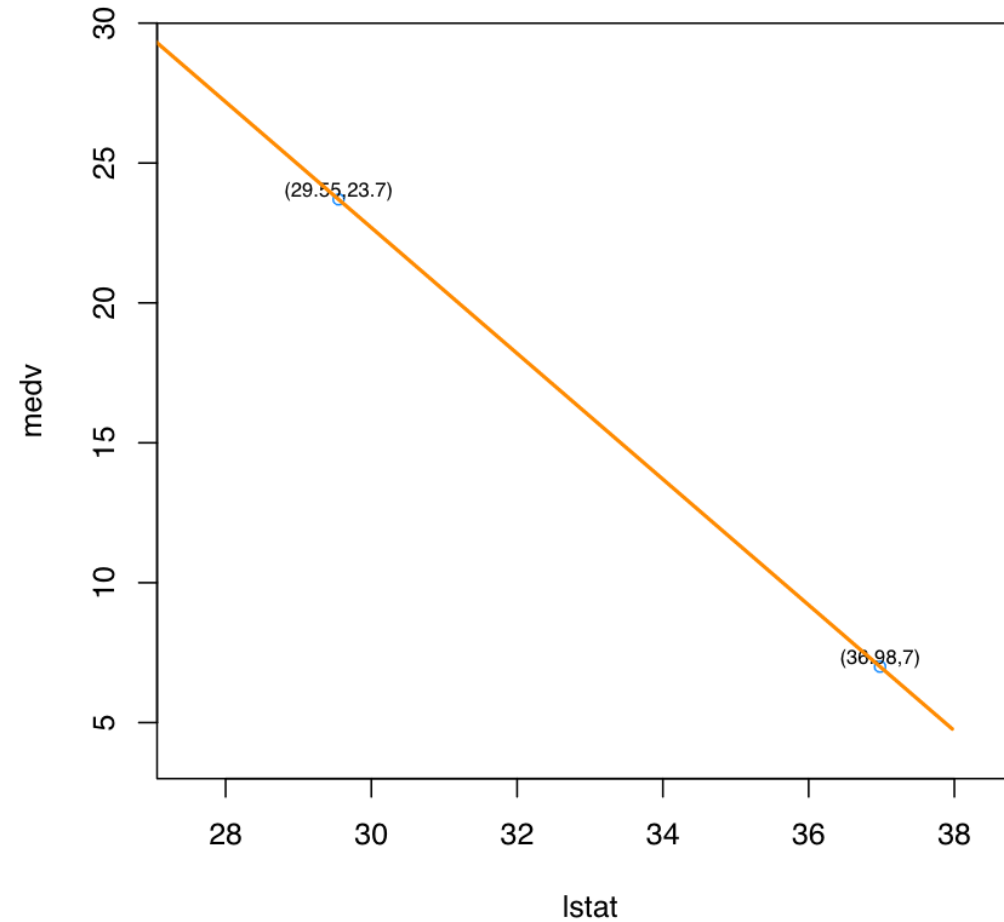
- Suppose we only have two observations ($n = 2$)

crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	lstat	medv
0.28955	0	10.59	0	0.489	5.412	9.8	3.5875	4	277	18.6	29.55	23.7
45.74610	0	18.10	0	0.693	4.519	100.0	1.6582	24	666	20.2	36.98	7.0

- Let us consider the same model: $medv = \beta_0 + lstat \cdot \beta_1 + \varepsilon$
- We can estimate β_0 and β_1 with two data points (solving a linear system)



Recap: Explaining overfitting

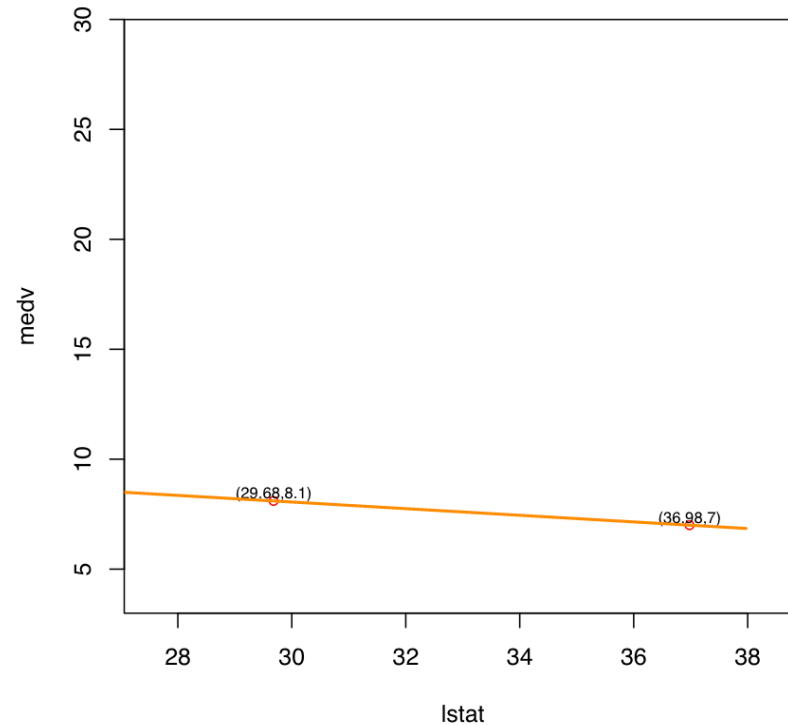


- Problem: The fitted curve is sensitive to the *medv* of these two observations



Recap: Explaining overfitting

- If one of the two observations changes, we can get a very different fitted curve



- This is an example of overfitting
- **Question: can you think of other examples of overfitting?**



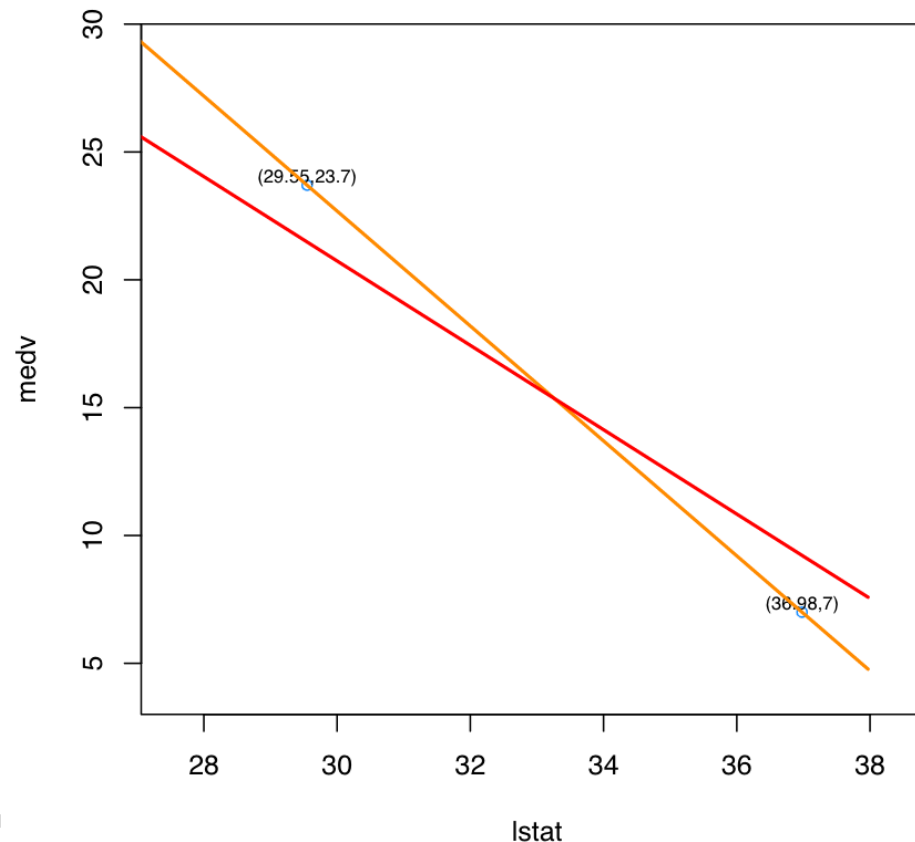
Applying weight decay

- Minimize
 - $\sum_{i=1}^n (\text{medv}_i - \beta_0 - \text{lstat}_i \cdot \beta_1)^2 + \lambda \cdot \beta_1^2$
 - $\lambda \geq 0$: tuning hyper-parameter
- Without weight decay: $\widehat{\text{medv}} = 90.118 - 2.248 \cdot \text{lstat}$
 - $\sum_{i=1}^n (\text{medv}_i - \hat{\beta}_0 - \text{lstat}_i \cdot \hat{\beta}_1)^2 + \lambda \cdot \hat{\beta}_1^2 = 0 + 10 \cdot 2.248^2 = 50.535$
 - Perfectly fitting the data incurs high loss



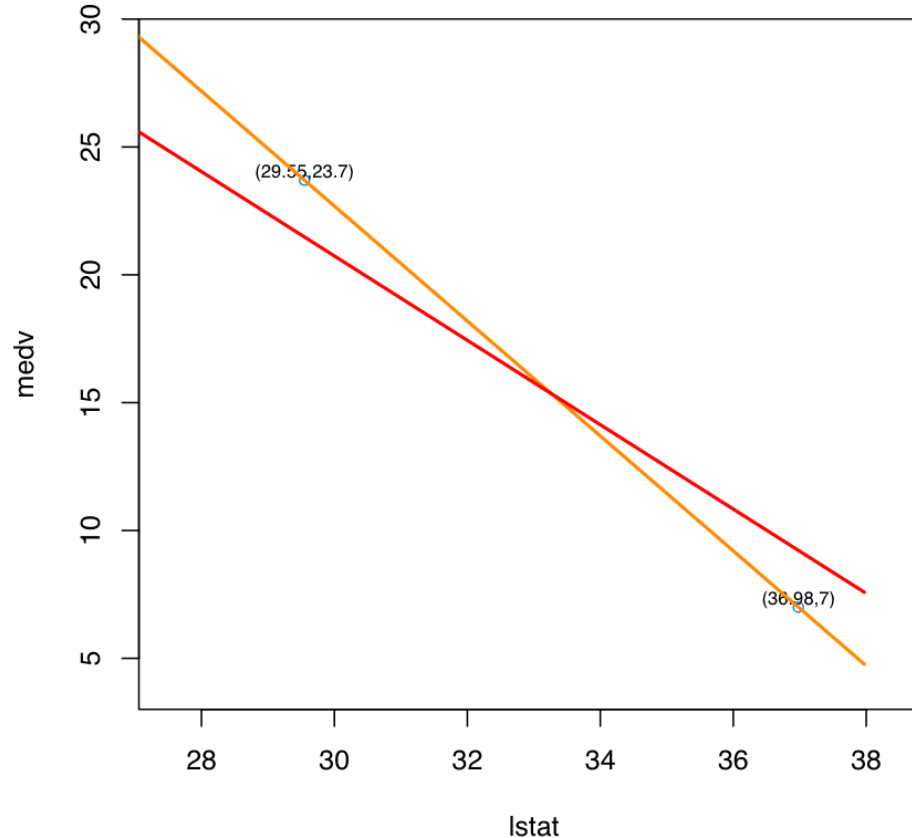
Applying weight decay

- Adjusted fit: $\widehat{medv} = 70.234 - 1.650 \cdot lstat$
 - $\sum_{i=1}^n (medv_i - \hat{\beta}_0 - lstat_i \cdot \hat{\beta}_1^R)^2 + \lambda \cdot (\hat{\beta}_1^R)^2 = 4.931 + 4.931 + \textcolor{red}{10} \cdot 1.650^2 = 37.084 < \textcolor{blue}{50.535}$



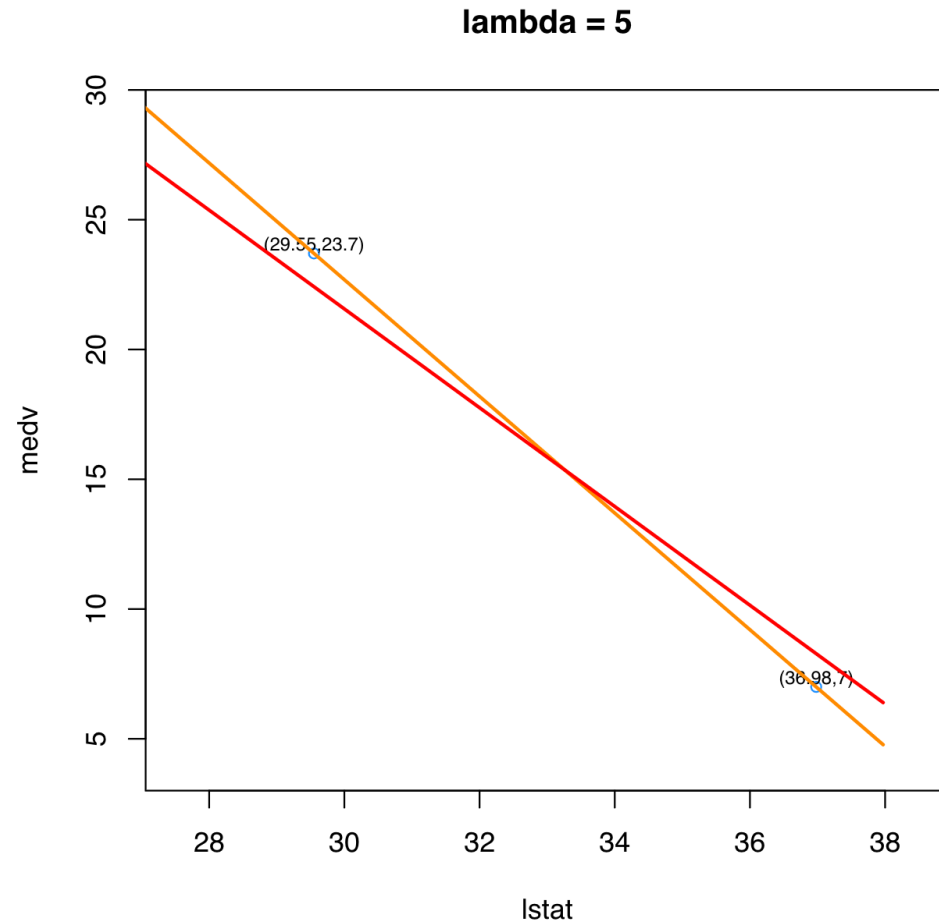
Weight decay reduces sensitivity

- Without weight decay, one unit change in *lstat* results in -2.248 units change in *medv*
- With weight decay, one unit change in *lstat* results in -1.650 units change in *medv*

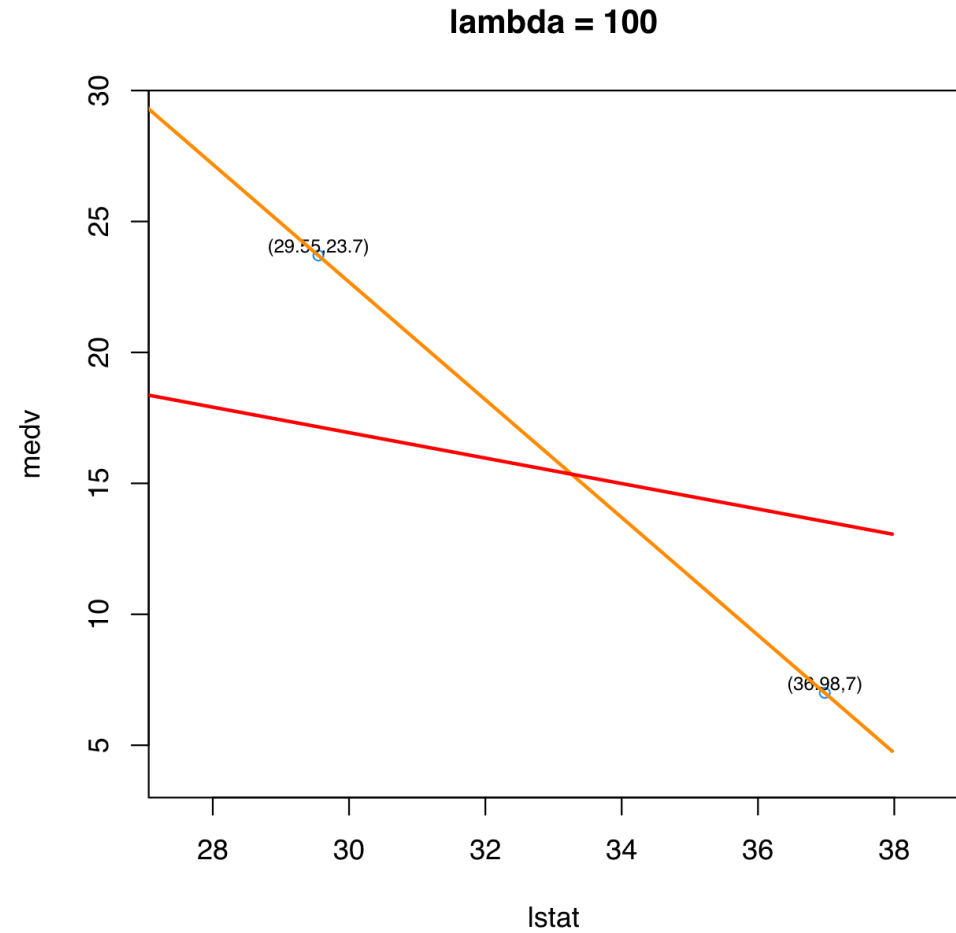


λ adjusts the sensitivity of the model

- $\lambda = 5$



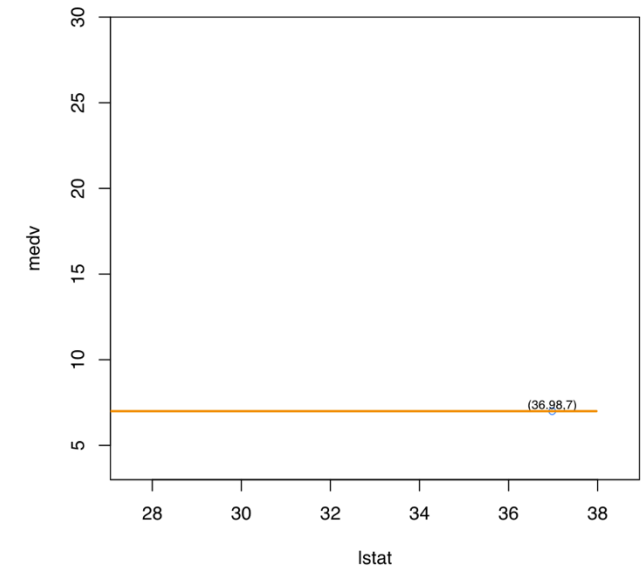
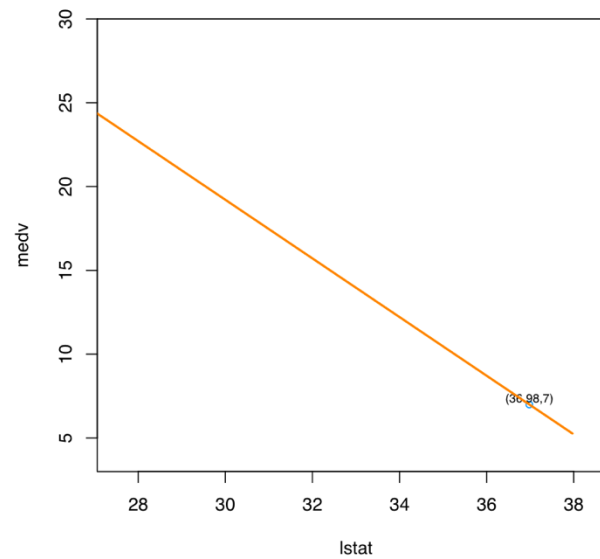
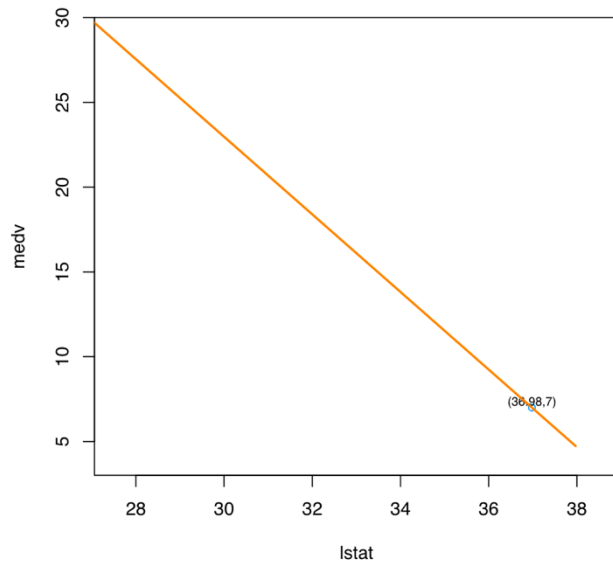
- $\lambda = 100$



Quiz: Which line is the ridge regression fit?

- One observation ($n = 1$)

crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	lstat	medv
45.7461	0	18.1	0	0.693	4.519	100	1.6582	24	666	20.2	36.98	7



Lecture plan

- **Introduction to language modeling I**
 - What is a language model?
 - Several historical language modeling paradigms
 - What are the capabilities of language models?



What is a language model?

- **Classical definition of a language model (LM):** a probability distribution over sequences of tokens

- Suppose we have a vocabulary V of a set of tokens
- A language model p assigns each sequence of tokens $x_1, x_2, \dots, x_L \in V$ a probability (a number between 0 and 1):

$$p(x_1, x_2, \dots, x_L)$$

- The probability tells us how “good” a sequence of tokens is
- For example, given vocabulary {ate, ball, cheese, mouse, the}, LM may assign:

$$p(\text{the, mouse, ate, the, cheese}) = 0.02$$

$$p(\text{the, cheese, ate, the, mouse}) = 0.01$$

$$p(\text{mouse, the, the, cheese, ate}) = 0.0001$$

- **Generation:** A language model p takes a sequence and returns a probability to assess its goodness, $x_{1:L} \sim p$



Autoregressive language models

- We can write the joint distribution $p(x_{1:L})$ of a sequence $x_{1:L}$ using chain rule of probabilities

$$p(x_{1:L}) = p(x_1) \cdot p(x_2|x_1) \cdot p(x_3|x_1, x_2) \cdots p(x_L|x_{1:L-1}) = \prod_{i=1}^L p(x_i|x_{1:i-1})$$

- For example:

$$p(\text{the, mouse, ate, the, cheese}) = p(\text{the}) \times p(\text{mouse}|\text{the}) \times p(\text{ate}|\text{the, mouse}) \times p(\text{the}|\text{the, mouse, ate})$$

- In particular, $p(x_i|x_{1:i-1})$ is a conditional probability distribution of the next token x_i given the previous tokens $x_{1:i-1}$
- An Autoregressive language model computes each conditional probability efficiently using a neural network



Generation

- To generate an entire sequence $x_{1:L}$ from an autoregressive LM p , sample one token at a time given tokens generated so far

$$\text{For } i = 1, \dots, L: x_i \sim p(x_i | x_{1:i-1})^{\frac{1}{T}}$$

- Where $T \geq 0$ is a temperature parameter that controls how much randomness we want from the LM
 - $T = 0$: deterministically choose the most probable token x_i at each position i
 - $T = 1$: sample “normally” from pure LM
 - $T = \infty$: sample from a uniform distribution over entire vocabulary
-
- To ensure the probability distribution sums to one, we re-normalize the distribution



Conditional generation

- Conditional generation: specify a prefix sequence $x_{1:i}$ (**prompt**) and sample the rest $x_{i+1:L}$ (**completion**)

the, mouse, ate → the, cheese



prompt completion



Summary

- A language model gives a probability distribution p over sequences $x_{1:L}$
- A good language model should have linguistic capabilities and world knowledge
- An autoregressive language model allows for efficient generation of a completion $x_{i+1:L}$ given a prompt $x_{1:i}$
- Use temperature to control amount of variability in generation



Transfer learning

- Transfer learning: use the information learned from one task to help learn another task
- Example #1: building a face recognition system from open-source models plus a few hundred labeled examples
- Example #2: fine-tuning a pre-trained language model for solving a downstream text prediction task
- Multitask learning: simultaneously train a multitask learning model on multiple objectives



Information theory

- Information theory, entropy of English, n-gram models
 - Language models date back to Claude Shannon; the **entropy** of a distribution is defined as

$$H(p) = \sum_x p(x) \log \frac{1}{p(x)}$$

- The entropy measures the expected number of bits any algorithm needs to encode a sample $x \sim p$ into a bit string:

the mouse ate the cheese $\Rightarrow$ 0001110101

- Lower entropy means the sequence has more “structure,” hence shorter code length
- Cross entropy: expected number of bits needed to encode a sample $x \sim p$ using a compression scheme given by model q

$$H(p, q) = \sum_x p(x) \log \frac{1}{q(x)}$$



N-gram models

- Tokenization: the process of dividing a text into a sequence of words
- N-gram models: the prediction of a token x_i only depends on the last $n - 1$ characters rather than the full history

$$p(x_i | x_{1:i-1}) = p(x_i | x_{i-(n-1):i-1})$$

- Unigram for 1-gram, bigram for 2-gram
- Or, a trigram ($n = 3$) model for 3-gram

$$p(\text{cheese} \mid \text{the, mouse, ate, the}) = p(\text{cheese} \mid \text{ate, the})$$



Neural language models

- With a neural network, we represent an N-gram model as

$$p(\text{cheese} \mid \text{ate, the}) = \text{NN}(\text{ate, the, cheese})$$

- Examples of NN:
 - Recurrent neural networks (RNNs)
 - Transformers (GPT-3 used $n = 2,048$)



Lecture plan

- **Introduction to language modeling I**
 - What is a language model?
 - Several historical language modeling paradigms
 - What are the capabilities of language models?



GPT (Pretraining)

- General pretrained transformers (GPT)
 - Train a language model to predict the next word
 - GPT models do a good job on a broad range of NLP tasks



Fine-tuning (Adaptation)

- Recall that a language model p is a distribution over sequences of tokens $x_{1:L}$: $p(\text{the, mouse, ate, the, cheese})$
- A **task** is a mapping from inputs to outputs:
- Question answering:
 - Input: In what school did Albus Dumbledore teach?
 - Output: The wizarding school Hogwarts
- Adaptation refers to the process of taking a language model and tuning it for another downstream task



Probing vs. fine-tuning

- Train a new model that maps inputs to outputs, by
 - Probing: Creating a new model that uses the language model as features
 - Fine-tuning: Starting with the language model and updating it based on training instances
 - Lightweight fine-tuning: adapters, low-rank adaptation



Prompting

- In-context learning: Construct a prompt (a string based on description and training instances) or a set of prompts, feed into a language model to obtain completions
 - Zero-shot learning: number of training examples is 0
 - One-shot learning: number of training examples is 1
 - Few-shot learning: number of training examples is few (less than a few hundred)
- Surprising findings of LLMs: Model is trained to predict the next masked token, but it performs well on a wide variety of downstream tasks
- Capable of in-context learning with just a few prompts



Language modeling

- A language model p takes a corpus of text $x_{1:L}$: decompose joint probability into the product of the conditional probabilities

$$p(x_{1:L}) = \prod_{i=1}^L p(x_i | x_{1:i-1})$$

- Perplexity: take the geometric average of the conditional probabilities

$$\text{perplexity}(x_{1:L}) = \exp \left(\frac{1}{L} \sum_{i=1}^L \log \frac{1}{p(x_i | x_{1:i-1})} \right)$$

- There are two types of errors a language model can make
 - Recall error: the model fails to place probability mass on some token
 $p(\text{ate} \mid \text{the, mouse}) \rightarrow 0 \Rightarrow \text{perplexity}(\text{the, mouse, ate, the, cheese}) \rightarrow 0$
 - Precision error: the model places extra probability mass on some bad sequences



Examples of NLP tasks

- Task: predict the last word of a sentence
 - Motivation: Solve the task requires long-range dependencies
 - Example: Prompt: Alice was friends with Bob. Alice went to visit her friend ____
 - Answer: Bob
- Input: a question, Output: an answer
 - TriviaQA: given a trivia question, generate the answer
 - Adaptation: Q: “Harry Potter” is authored by?
 - A: J. K. Rowling



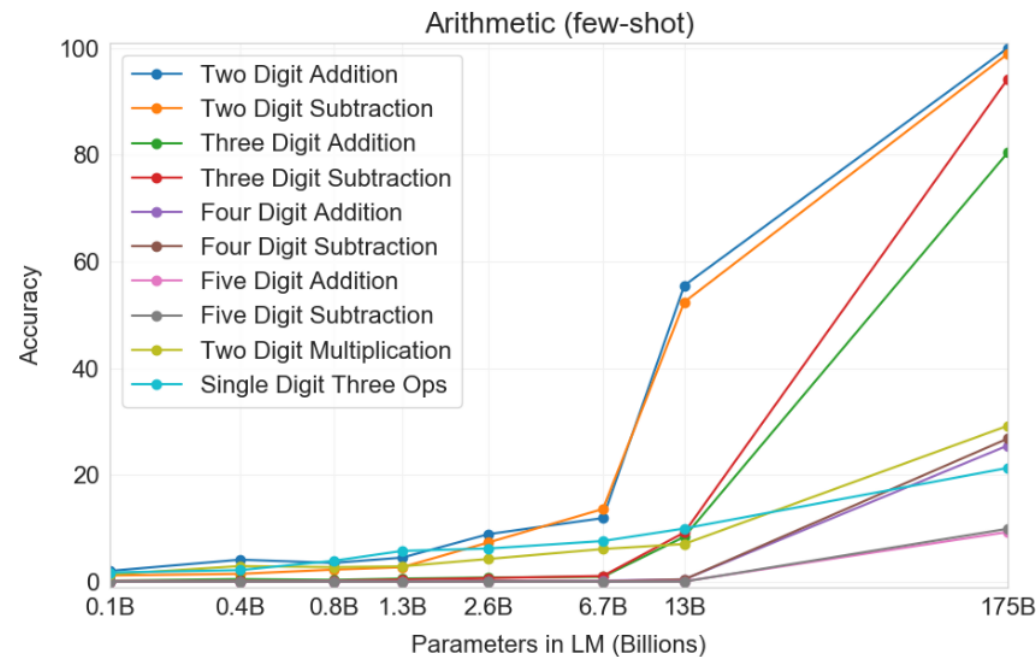
Machine translation

- Translate a sentence in a source language (e.g., German) to sentence in a target language (e.g., English)
- Evaluation: Since there are multiple possible translations, the (automatic) evaluation metric is BLEU
 - Captures a notion of n-gram overlap
 - <https://en.wikipedia.org/wiki/BLEU>



Arithmetic/Math reasoning tasks

- GPT-3 is a language model, but we can evaluate it on a range of more abstract reasoning tasks
- Task: do arithmetic. Example: $a^2 + ab + b^2 \pmod{p}$, where p is a prime number



News article generation

- Task: given title and subtitle, generate a news article
 - Evaluation: human rated articles based on how likely the article is written by a machine
- Other tasks
 - SWORDS: lexical substitution, where the goal is to predict synonyms in the context of a sentence
 - Massive multitask language understanding: 57 multiple-choice problems spanning mathematics, US history, computer science, etc
 - Truthful QA: question answering dataset that humans would answer falsely due to misconceptions

